

Week 2 - Day 6b: Operations and Data Types

You will be working on a series of exercises for the material covered in lecture 6b, including:

- **While vs For Loops**
- **Nested Loops**
- **Input**
- **Modules**

We will do a few exercises to solidify your understanding on these concepts.

Exercise 1: While vs For Loops []

Given the following function that prints the first `n` even natural numbers using a `while` loop:

```
def smaller_evens(n):  
    i = 1  
    while i <= n:  
        print(2 * i)  
        i += 1
```

1.1 Implement this function using a `for` loop.

```
# TODO: Implement the above function using a "for" loop
```

Imagine you are walking through **Merkato**, Addis Ababa's famous marketplace, and you're visiting stalls 1 to 10.

- Stall 5: The seller is missing, **skip** it.
- Stall 8: The stall is closing early, you have to **stop** your trip.
- All other stalls: Print: "You visited stall number X in Merkato"

1.1.2 Implement the above logic with a loop using `continue` and `break`.

```
# TODO: You code here.
```

Consider the following function that returns the index of the first occurrence of an item in a list using a `for` loop:

```
def linear_search(lst, item):  
    for i in range(len(lst)):  
        if lst[i] == item:  
            return i  
    return -1
```

1.2 Implement this function using a `while` loop.

TODO: Implement the above function using a "while" loop

The following function counts how many times you need to add random numbers (0s or 1s) until their sum `rand_sum` reaches a specified number `n`:

```
import random
def count_random_bits(n):
    rand_sum = 0
    i = 0
    while rand_sum < n:
        rand_sum += random.randint(0,1)
        i += 1
    return i
```

1.3 Rewrite this function to use a `for` loop instead of a `while` loop. Do not use the `break` keyword.

TODO: Try to implement the above function using a "for" loop

1.4 Don't worry if you couldn't. Why were you not able to implement it using a `for` loop?

1.5 Write a function `find_freq(L, n)` that returns the frequency of a number `n` in a list `L` using a `while` loop.

Example: For the input `L = [6, 6, 7, 8, 7, 8, 6, 6, 8]` and `n = 6`, the output should be 4 because 6 appears 4 times in `L`.

TODO: Implement the function here

1.6 Write a function that solves the problem described in question 1.5 using a `for` loop instead of a `while` loop.

TODO: Implement the function here

1.7 Which one do you think is easier to implement? 1.5 or 1.6? Why?

1.8 Write a function that takes a list of integers `L` (which may contain duplicates) and returns a list of all the unique elements in `L`. Do not use any built-in Python functions.

Example: If `L = [1, 1, 1, 2, 2, 3, 3, 3, 3]`, the output should be `[1, 2, 3]`.

TODO: Write your code here.

1.9 The [Leibniz formula](#) for π is an infinite series representation that expresses $\frac{\pi}{4}$ as the sum of an alternating series of fractions. It states that:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \dots$$

Which can be simplified as:

$$\pi = \frac{8}{1 \cdot 3} + \frac{8}{5 \cdot 7} + \frac{8}{9 \cdot 11} + \dots$$

Write a function `approximate_pi(n)` that approximates π using the above simplified infinite series. The function should keep adding terms until the new term is less than 10^{-n} .

You can compare your result with the actual value of π from the `math` module by using `math.pi`.

Note: This function can be quite slow, so try it for values of n less than 14.

1.9 The Leibniz formula for π is an infinite series representation that expresses $\frac{\pi}{4}$ as the sum of an alternating series of fractions. It states that:

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} \dots$$

$$\pi = \frac{8}{1 \cdot 3} + \frac{8}{5 \cdot 7} + \frac{8}{9 \cdot 11} + \dots$$

Write a function `approximate_pi(n)` that approximates π using the above simplified infinite series. The function should keep adding terms until the new term is less than 10^{-n} . You can compare your result with the actual result of π , which you can get from the `math` module by typing `math.pi`.

i This function is quite slow, so try it for an n less than 14.

```
# TODO: Implement the function that approximates pi.
import math

3.141592653589793
```

Exercise 2: Nested Loops []

2.1 Given a list of lists containing 0s and 1s, write a function that prints `.` when there is a 0 and `#` when there is a 1.

For example, given the input:

```
[
    [0, 1, 0, 0, 0, 1, 0],
    [0, 0, 0, 0, 0, 0, 0],
    [1, 0, 0, 0, 0, 0, 1],
    [0, 0, 1, 1, 1, 0, 0]
]
```

Your code should print:

```
.#...#.
.....
#.....#
..###..
```

Hint: `print("#", end="")` prints `"#"` without adding a new line; the cursor stays on the same line immediately after the `"#"`.

#TODO: Implement your function here

2.2 You are given a 2D list of integers. Write a function that returns a transformed list by multiplying each element of the input list by a power of 10. You should multiply the elements of the first row by 10^1 , the second row by 10^2 , the third row by 10^3 , and so on.

Example:

- Input: `[[5, 6], [3, 4, 6]]`
- Output: `[[50, 60], [300, 400, 600]]`

#TODO: Implement your code here

2.3 Given two integer arrays `nums1` and `nums2`, implement a function that returns an array of elements in `nums1` that also exist in `nums2`.

Example:

- Input: `nums1 = [5, 3, 7, 7, 9]`, `nums2 = [3, 3, 4, 4, 7, 8]`
- Output: `[3, 7, 7]`

Explanation:

- 5 does not exist in `nums2`
- 3 exists in `nums2`
- 7 exists in `nums2`
- 7 exists in `nums2`
- 9 does not exist in `nums2`

#TODO: Implement your code here

2.4 Write a function that takes a 2D list and returns a 1D list.

Example:

- Input: `[[50, 60], [300, 400]]`
- Output: `[50, 60, 300, 400]`

#TODO: Implement your code here

Exercise 3: Input []

3.1 Write a function `name_checker` that prompts the user to input their name and:

- Returns `True` if the input only contains letters of the alphabet and spaces.
- Prompts the user to input their name again if the input contains invalid characters.
- Returns `False` if the input is `"quit"`.

Use the helper function `isalphaspace(s)` to check whether the input is valid or invalid. `isalphaspace(s)` returns `True` if all characters in `s` are alphabetic or spaces, and `False` otherwise.

Hint: Think carefully about which kind of loop to use.

```
# TODO: Write your code

def isalphaspace(s):
    for char in s:
        if not (char.isalpha() or char.isspace()):
            return False
    return True

def name_checker():
    ...
```

3.2 Define a dictionary named `menu` where the keys are your favorite foods and the values are their prices.

```
# TODO: Define the dictionary "menu" here.
```

3.3 Write a function that:

- Displays the menu to the user.
- Asks the user to type the name of a food they want to order.
- Returns the price of the food if the input is valid.
- Keeps asking until the user inputs a valid food name from the `menu` dictionary or types `'quit'` to stop.

```
# TODO: Implement the function here
```

Exercise 4: Modules []

4.1 Use the `sqrt` function from the `math` module to calculate the square root of a number. Define an integer `n` and calculate its square root.

```
# TODO: Calculate the square root of "n" using the "sqrt" function
```

4.2 Write a function that generates and prints a random integer between 1 and 100.

```
# TODO: Write your code here
```

4.3 Write a function that generates and prints a random float number between 1 and 100.

Hint: Use the `random.uniform()` function.

```
# TODO: Write your code here
```

4.4 Write a function that uses the `ceil` and `floor` functions from the `math` module to round a number `n` to the closest integer.

- The `ceil` function returns the smallest integer value greater than or equal to the specified number.
- The `floor` function returns the largest integer value less than or equal to the specified number.

Example:

- When `n = 2.4`, the output should be 2.
- When `n = 2.7`, the output should be 3.

```
# TODO: Write your code here.
```

4.5 An identity function is a function that returns the same value which was used as its argument. If f is a function, then the identity relation for argument x is represented as $f(x)=x$ for all values of x . Implement this function for integers using the `log` and `pow` functions from the `math` module. You may need to use the rounding function described above.

```
# TODO: Write your code here.
```

Congratulations 